INF 201 - Deliverable_2

Introduction

This notebook contains Python code and explanations.

G2_2 Filmon Tsegay Kahsay and Amin Elanbri

Link to github: https://github.com/Amin12343/INF201-Deliverable-2

In [69]:
```
import json
import re
from pathlib import Path

import numpy as np
import pandas as pd
```

Task 1: File ingestion and regular expressions

In [70]:
```
def read_log_file(file_path: Path | str) -> tuple[str, str]:
    """Read a text file that is encoded with either UTF-8 or ISO-8859-1

    Args:
        file_path: Path to the file that will be read

    Returns:
        A tuple with the text content of the file and the encoding used.
    """
    file_path = Path(file_path) # I make sure that I have a Path object
    raw_bytes = file_path.read_bytes() #reads the whole file as bytes

    try: # try UTF-8 first
        text = raw_bytes.decode("utf-8") #here i turn the bytes into text
        encoding_used = "utf-8" #remember which encoding worked
    except UnicodeDecodeError: # the bytes are not valid UTF-8
        text = raw_bytes.decode("iso-8859-1") # here i use Latin-1 instead
        encoding_used = "iso-8859-1" #remembers which encoding that worked

    return text, encoding_used #returns both as a tuple
```

In [71]:
```
# Regular expression for one valid sales line, with one named group per field
LOG_LINE_PATTERN = re.compile(
    r"\[(?P<timestamp>\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2})\] " # [2026-10-15 08:
    r"\[(?P<register_id>\w+)\] " # [POS_01]
    r"item=(?P<item>\w+) " # item=Latte
    r"price=(?P<price>\d+\.\d+) " # price=4.50
    r"(?P<currency>[A-Z]{3}) " # EUR, NOK or USD
    r"\((?P<status>SUCCESS|REFUND|CANCELLED)\)$" # (SUCCESS) at the end of the l
)
```

In [72]:
```python
def parse_log(text: str) -> list[dict[str, str | float]]:
    """Extract all valid sales transactions form the text of the lig file.

    Lines that do not match the valid line format (system noise) are ignored

    Args:
        text: The text content of a log file

    Retuns:
        List with one dict per valid transaction, with thw keys timestamp,
        register_id, item, price, currency and status
    """
    transactions = [] # her i make a list that will hold one dict per transactio
    for line in text.splitlines(): #loops throug each line in the text
        match = LOG_LINE_PATTERN.match(line.strip()) #try to match the pattern
        if match is None: #the line is noise, not a sale line
            continue
        record = match.groupdict() #dict with the named groups as keys
        record["price"] = float(record["price"]) #price from text to float
        transactions.append(record) #adds the transaction to the list

    return transactions #retuns all valid transactions
```

In [73]:
```python
# test on one of the generated log files
test_file = Path("data/2026/library_cafe/POS_01/register_sales.log")
test_text, test_encoding = read_log_file(test_file) #reads the log
test_transactions = parse_log(test_text) #find the valid lines

print(f"Encoding used: {test_encoding}") #wich encoding eas used
print(f"Number of lines in file: {len(test_text.splitlines())}") #checks all lin
print(f"Number of valid transactions: {len(test_transactions)}") #valid lines on
print(test_transactions[0]) #shows the first transaction

#test that noise lines are ignored
noise_text = (
    "--- PRINTER PAPER LOW WARNING ---\n"
    "[DEBUG] BARCODE SCANNER REBOOTED\n"
    "[2026-10-15 08:30:00] [POS_01] item=Latte price=4.50 EUR (SUCCESS)\n"
)
assert len(parse_log(noise_text)) == 1 #only the sales line should be found
print("Noise lines are ignored correctly")
```

```
Encoding used: utf-8
Number of lines in file: 92
Number of valid transactions: 80
{'timestamp': '2026-10-15 07:01:47', 'register_id': 'POS_01', 'item': 'Cappuccin
o', 'price': 4.41, 'currency': 'USD', 'status': 'SUCCESS'}
Noise lines are ignored correctly
```

Results and discussion

The function read_log_file() read the test file as UTF-8. It tries UTF-8 first and only uses ISO-8859-1 if UTF-8 fails, because ISO-8859-1 never gives an error. The file was read as UTF-8 even though some files are written with ISO-8859-1. This is because the logs only contain ASCII characters, which are the same in both encodings.

The test file had 90 lines, and parse_log() found 80 valid transactions. The other 10 lines were system noise. The regular expression only matches the exact format of a sales line, so noise lines are ignored, including [DEBUG] lines that also start with a bracket.

Task 2 Recursive utility functions

In [74]:
```python
def find_log_files(folder: Path | str, extension: str = ".log") -> list[Path]:
    """Find all files with the given extension in a folder and all its subfolder

    The function calls itself for each subfolder it finds (recursion)

    Args:
        folder: Path to the folder to search in
        extension: File extension to look for (default ".log")

    Returns:
        List of Path objects for all matching files
    """
    folder = Path(folder) #makes sure that i have a Path object
    log_files = [] #list that will hold the paths to the log files

    for item in sorted(folder.iterdir()): #loop through everything in the folder
        if item.is_dir(): #the item is a subfolder
            log_files = log_files + find_log_files(item, extension) #searching i
        elif item.is_file() and item.name.endswith(extension): # a matching file
            log_files.append(item) #add the file to the list

    return log_files #return all log files that were found
```

In [75]:
```python
def read_store_config(config_path: Path | str) -> dict:
    """Read the store configuration from JSON file

    Args:
        Config_path: Path to the JSON file

    Returns:
        The content of the files as a Python dictionary
    """
    with open(config_path, encoding="utf-8") as f: #open the file and close it a
        return json.load(f) # turn the json content into a dictonary
```

In [76]:
```python
def find_active_registers(data: dict | list) -> set[str]:
    """Find the IDs of all active registers in nested dictionaries and lists

    The function calls itself for every dictionary and list it finds (recursion)

    Args:
        data: Any combanation of nested dictionaries and lists

    Returns:
        A set with the IDs of all registers where "active" is true
    """
    active_ids = set() #empty set that will hold the active register IDs

    if isinstance(data, dict): #the data is a dictionary
        if "id" in data and "active" in data and data ["active"] is True:
            active_ids.add(data["id"]) #add its ID ti the set
```

```
        for value in data.values(): #loop through all values in the dictionary
            active_ids = active_ids | find_active_registers(value) #searching in

    elif isinstance(data, list): #the data is a list
        for element in data: # loop trough all elements in the list
            active_ids = active_ids | find_active_registers(element) #search ins

    return active_ids #returning alll active IDs that were found
```

In [77]:
```
log_files = find_log_files("data") #find all log files in the data folder
print(f"Number of log files found: {len(log_files)}")

store_config = read_store_config("data/store_config.json") #here I make it read
active_registers = find_active_registers(store_config) #finding the active regis
print(f"Active registers: {active_registers}")
```

```
Number of log files found: 9
Active registers: {'POS_03', 'POS_01', 'POS_02'}
```

Results and discoussion

The function find_log_files() found all 9 log files by calling itself for every subfolder, without using glob() or os.walk(). the function find_active_registeres() found that POS_01, POS_02 and POS_03 are active, while POS_04 was left out. Because bith functions use recurision, they work no matter how deeply the folders or data are nested.

Task 3: Functions for cleaning and analysing data using NumPy

In [78]:
```
def remove_price_outliers(prices: list[float] | np.ndarray) -> np.ndarray:
    """Replace price outliers with the median of the valid prices

    A price is an outlier if it is more than 3 standard deviations
    away from the mean: |x - mean| > 3 * std

    Args:
        prices: the prices to clean, as a list or aa Numpy array

    Returns:
        A 1D numpy float array where the outliers are replaced by the median
    """
    prices = np.array(prices, dtype=float) #convert to a 1D numpy float array
    mean_prices = np.mean(prices) #the mean of all prices
    std_price = np.std(prices) #the standard deviation of all prices

    distance = np.abs(prices - mean_prices) # Distance form the mean for each pr
    is_outlier = distance > 3 * std_price #True for prices that are outliers
    is_valid = distance <= 3 * std_price #true for prices that are not outliers

    median_valid = np.median(prices[is_valid]) #median of the valid prices only

    cleaned_prices = prices.copy() # I use copy() so the original array is not c
    cleaned_prices[is_outlier] = median_valid # here i rplace the outliers with
    return cleaned_prices # return the cleaned array
```

In [79]:
```
EPSILON = 1e-6 #samll number added to avoid division by zero


def compute_refund_ratio(sales_count: np.ndarray, refund_count: np.ndarray) -> n
    """Compute the refund ratio element-wise: refund_count / (sales_count + epsi

    Args:
        sales_count: Number of sales for each item
        refund_count: number of refunds for each time

    Returns:
        A numpy array with the refund ratio for each item
    """
    sales_count = np.array(sales_count, dtype=float) #make sure er have a float
    refund_count = np.array(refund_count, dtype=float)
    return refund_count / (sales_count + EPSILON) #element-wise refund ratio
```

In [80]:
```
#collect all prices from all log files (using the functions from task 1 and 2)
all_prices = [] #list that will hold all prices
for log_file in find_log_files("data"): #loop through all log files
    log_text, _ = read_log_file(log_file) #reads the file and ignores the encodi
    for transaction in parse_log(log_text): #loops throough the valid transactio
        all_prices.append(transaction["price"]) #adding the price to the list

cleaned_prices = remove_price_outliers(all_prices) #removing the outliers
print(f"Highest price before cleaning: {max(all_prices):.2f}")
print(f"Highest price after cleaning: {np.max(cleaned_prices):.2f}")

#test the refund ratio on three example items: 2/10, 0/0 an 5/5
# the second item has 0 sales, which shows that epsilon prevents division by zer
print(compute_refund_ratio(np.array([10, 0, 5]), np.array([2, 0, 5])))
```

```
Highest price before cleaning: 999.00
Highest price after cleaning: 7.98
[0.19999998 0.         0.9999998 ]
```

Results and discussion

The function remove_price_outliers() found the hardware glitches in the prices. The highest price was 999.00 before cleaning and 7.99 after cleaning, so all outliers were replaced. We used the median of the valid prices because, unlike the mean, it is not pulled up by the extreme values.

The function compute_refund_ratio() returned about 0.2, 0.0 and 1.0 for the three test items. The second item had 0 sales, and epsilon made sure the result was 0 instead of a division-by-zero error.

Task 4: Data analysis using Pandas

In [81]:
```
def create_dataframe(transactions: list[dict]) -> pd.DataFrame:
    """
    Convert transaction records into a chronologically sorted DataFrame.
    Args:
        transactions: List of transaction dictionaries.
```

```
    Returns:
           DataFrame with a sorted datetime index.
    """


    df = pd.DataFrame(transactions)
    df["timestamp"] = pd.to_datetime(df["timestamp"])
    df = df.set_index("timestamp")
    df = df.sort_index()


    return df
```

In [82]:
```
df = create_dataframe(test_transactions)
print(df.head())
```

```
                     register_id        item  price currency     status
timestamp
2026-10-15 07:01:47      POS_01  Cappuccino   4.41      USD    SUCCESS
2026-10-15 07:02:42      POS_01    Sandwich   7.23      NOK  CANCELLED
2026-10-15 07:04:22      POS_01      Muffin   3.54      EUR    SUCCESS
2026-10-15 07:05:40      POS_01    Espresso   2.52      EUR  CANCELLED
2026-10-15 07:08:35      POS_01    Sandwich   7.77      NOK    SUCCESS
```

In [83]:
```
def hourly_revenue(df: pd.DataFrame) -> pd.Series:
    """
    Calculate total transaction prices per item in hourly intervals.

    Args:
        df: DataFrame containing transaction data.

    Returns:
        Series with hourly total prices for each item.
    """
    result = df.groupby("item").resample("1h")["price"].sum()
    return result
```

In [84]:
```
hourly_result = hourly_revenue(df)
print(hourly_result.head(10))
```

```
item        timestamp
Cappuccino  2026-10-15 07:00:00     24.45
            2026-10-15 08:00:00   1032.15
            2026-10-15 09:00:00      4.26
Croissant   2026-10-15 07:00:00     11.35
            2026-10-15 08:00:00      8.30
            2026-10-15 09:00:00      8.47
Espresso    2026-10-15 07:00:00     23.05
            2026-10-15 08:00:00     17.37
            2026-10-15 09:00:00      2.75
Latte       2026-10-15 07:00:00     16.63
Name: price, dtype: float64
```

In [85]:
```
def create_status_pivot(df: pd.DataFrame) -> pd.DataFrame:
    """
    Create a pivot table of total transaction prices by item and status.

    Args:
        df: DataFrame containing transaction data.
```

```
    Return:
        DataFrame with total prices for each item and status.
    """
    result = df.pivot_table(
        index="item",
        columns="status",
        values="price",
        aggfunc="sum"
    )
    return result
```

In [86]:
```
pivot_result = create_status_pivot(df)
print(pivot_result)
```

```
status      CANCELLED  REFUND  SUCCESS
item
Cappuccino       7.89    4.15  1048.82
Croissant        3.82     NaN    24.30
Espresso         7.72    5.40    30.05
Latte            8.29   13.92    31.79
Muffin          11.41   14.02  1033.09
Sandwich        45.32     NaN    45.82
```

In [87]:
```
def item_refund_ratios(df: pd.DataFrame) -> pd.DataFrame:

    """
    Calculate the refund ratio for each item using NmPy.

    Args:
       df: DataFrame containing transaction data.

    Returns:
        DataFrame containing the refund ratio for each item.
    """
    counts = df.groupby(["item", "status"]).size().unstack(fill_value=0)


    sales_count = counts["SUCCESS"].to_numpy()
    refund_count = counts["REFUND"].to_numpy()


    ratios = compute_refund_ratio(sales_count, refund_count)


    result = pd.DataFrame({
        "refund_ratio": ratios
        }, index=counts.index)

    return result
```

In [88]:
```
refund_result = item_refund_ratios(df)
print(refund_result)
```

```
            refund_ratio
item
Cappuccino      0.076923
Croissant       0.000000
Espresso        0.200000
Latte           0.428571
Muffin          0.363636
Sandwich        0.000000
```

Results and discussion

The transaction data was converted into a DataFrame and sorted chronologically. Hourly transaction totals and pivot table were genereated for each item. Due to oulieres some prices were unsually high. The refund ratios varied between products, the highest ratio was 42.86% for Latte, our results works as expected.

Task 5: Full workflow

In [89]:
```
# Find all log files recursvely
log_files = find_log_files("data")
print(len(log_files))

# Read configuration and identify active registers
config = read_store_config("data/store_config.json")
active_registers = find_active_registers(config)
print(active_registers)
```

```
9
{'POS_03', 'POS_01', 'POS_02'}
```

In [90]:
```
# Read and parse all log files
all_transactions = []

for file in log_files:
    text, encoding = read_log_file(file)
    transactions = parse_log(text)
    all_transactions.extend(transactions)

# Check the total number of transactions
print(len(all_transactions))
```

```
720
```

In [91]:
```
# Convert transactions into a DataFrame
df_all = create_dataframe(all_transactions)

# Remove price outliers
cleaned_prices = remove_price_outliers(df_all["price"])
df_all["price"] = cleaned_prices

# Convert NOK prices using the requird factor of 10
df_all.loc[df_all["currency"] == "NOK", "price"] *= 10.0
```

In [92]:
```
# Calculate hourly revenue for each item
hourly_result = hourly_revenue(df_all)
```

```
# Creat a pivot table showing prices by item and status
pivot_result = create_status_pivot(df_all)

#Calculate refund ratios for each item
refund_result = item_refund_ratios(df_all)
```

In [93]:
```
# Save the cleaned transactions as a CSV file
df_all.to_csv("cleaned_transactions.csv")

#Save the pivot table and refund ratios as CSV files
pivot_result.to_csv("status_pivot_table.csv")
refund_result.to_csv("item_refund_ratios.csv")

# Save the hourly revenue report as JSON file
hourly_df = hourly_result.reset_index()
hourly_df.to_json("hourly_revenue.json", orient="records", date_format="iso")
```

In [94]:
```
files = [
    "cleaned_transactions.csv",
    "hourly_revenue.json",
    "status_pivot_table.csv",
    "item_refund_ratios.csv"
]

# Check that all output files exist
for file in files:
    print(file, Path(file).exists())
```

```
cleaned_transactions.csv True
hourly_revenue.json True
status_pivot_table.csv True
item_refund_ratios.csv True
```

Result and Discussion

9 log files were found containing 720 transactions, then the transactions were combined into a Pandas DataFrame and sorted by timestamp. We used function from Task 3 to correct price outliers, and prices in NOK were multiplied by 10. To generate hourly revenue, a status pivot table and refund ratios for each item we used the cleaned data. At the end all four output files were successfully saved and verified to exist.